

Advanced Deep Learning

Introduction to flow based generative models

Lucas Drumetz

IMT Atlantique

Feb, 6th, 2025

Flow Matching / Stochastic Interpolants

Preprint

FLOW MATCHING FOR GENERATIVE MODELING

Yaron Lipman^{1,2} Ricky T. Q. Chen¹ Heli Ben-Hamu² Maximilian Nickel¹ Matt Le¹
¹Meta AI (FAIR) ²Weizmann Institute of Science

ABSTRACT

We introduce a new paradigm for generative modeling built on Continuous Normalizing Flows (CNFs), allowing us to train CNFs at unprecedented scale. Specifically, we present the notion of Flow Matching (FM), a simulation-free approach for training CNFs based on regressing vector fields of fixed conditional probability paths. Flow Matching is compatible with a general family of Gaussian probability paths for transforming between noise and data samples—which subsumes existing diffusion paths as specific instances. Interestingly, we find that employing FM with diffusion paths results in a more robust and stable alternative for training diffusion models. Furthermore, Flow Matching opens the door to training CNFs with other, non-diffusion probability paths. An instance of particular interest is using Optimal Transport (OT) displacement interpolation to define the conditional probability paths. These paths are more efficient than diffusion paths, provide faster training and sampling, and result in better generalization. Training CNFs using Flow Matching on ImageNet leads to consistently better performance than alternative diffusion-based methods in terms of both likelihood and sample quality, and allows fast and reliable sample generation using off-the-shelf numerical ODE solvers.

1 INTRODUCTION

Deep generative models are a class of deep learning algorithms aimed at estimating and sampling from an unknown data distribution. The recent influx of amazing advances in generative modeling, e.g. for image generation Ramesh et al. (2022); Rombach et al. (2022), is mostly facilitated by

Journal of Machine Learning Research 26 (2025) 1-80

Submitted 12/23; Revised 9/25; Published 9/25

Stochastic Interpolants: A Unifying Framework for Flows and Diffusions

Michael S. Albergo*
Center for Cosmology and Particle Physics
New York University
New York, NY 10012, USA

ALBERGO@NYU.EDU

Nicholas M. Boffi*
Courant Institute of Mathematical Sciences
New York University
New York, NY 10012, USA

BOFFI@CIMS.NYU.EDU

Eric Vanden-Eijnden
Courant Institute of Mathematical Sciences
New York University
New York, NY 10012, USA

EVE2@CIMS.NYU.EDU

Editor: Maxim Raginsky

Abstract

A class of generative models that unifies flow-based and diffusion-based methods is introduced. These models extend the framework proposed in Albergo and Vanden-Eijnden (2023), enabling the use of a broad class of continuous-time stochastic processes called ‘stochastic interpolants’ to bridge any two probability density functions exactly in finite time. These interpolants are built by combining data from the two prescribed densities with an additional latent variable that shapes the bridge in a flexible way. The time-dependent probability density function of the stochastic interpolant is shown to satisfy a first-order transport equation as well as a family of forward and backward Fokker-Planck equations with tunable diffusion coefficient. Upon consideration of the time evolution of an individual sample, this viewpoint immediately leads to both deterministic and stochastic generative models based on probability flow equations or stochastic differential equations with an adjustable level of noise. The drift coefficients entering these models are time-dependent velocity fields characterized as the unique minimizers of simple quadratic objective functions, one of which is a new objective for the score of the interpolant density. We show that minimization of these quadratic objectives leads to control of the likelihood for generative models built upon

arXiv:2303.08797v4 [cs.LG] 9 Oct 2025

2747v2 [cs.LG] 8 Feb 2023

<https://arxiv.org/abs/2210.02747>

<https://arxiv.org/pdf/2303.08797>

Outline

Generative models

- Problem statement

- Static models

Flow based models

- The change of variables formula

- Normalizing flows

Flow matching

- The continuity equation

- Flow matching

Lecture Organization

- ▶ Two days (4 slots each) on generative modeling
- ▶ Day 1: Lecture
- ▶ Day 2: MCQ, End of Lecture, and Lab
- ▶ MCQ and lab will be part of the evaluation

Table of Contents

Generative models

- Problem statement

- Static models

Flow based models

- The change of variables formula

- Normalizing flows

Flow matching

- The continuity equation

- Flow matching

Problem statement

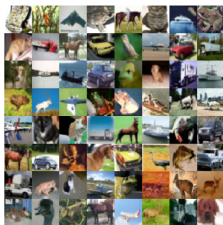
Given n samples $x_1, \dots, x_n$ drawn (i.i.d) from random variable $X \in \mathbb{R}^d$ with probability density (pdf) p_X ,

Design an algorithm allowing to draw new samples distributed according to p_X

→ the target distribution is known through N samples only.

Applications

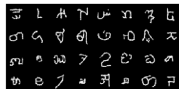
- Image or text generation (unconditional: focus of this lecture)



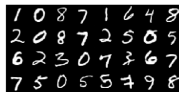
(a) CIFAR-10



(b) CelebA-HQ-256



(c) OMNIGLOT

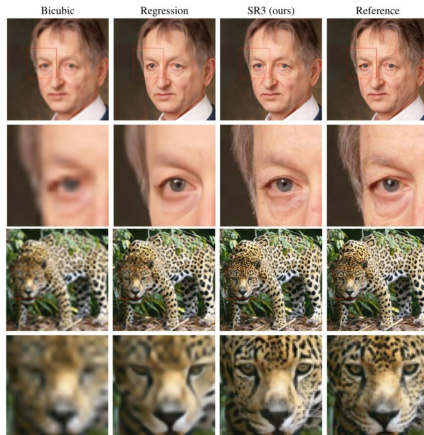


(d) MNIST

Vahdat, A., Kreis, K., & Kautz, J. Score-based generative modeling in latent space. NeurIPS 2021

Applications

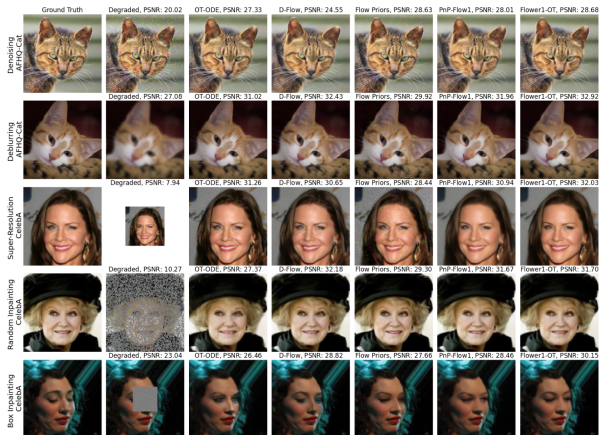
- ▶ Conditional generation: sample from $p(x|y)$ for some context y
 - ▶ Text or image generation from text: LLMs and image generation models from prompts
 - ▶ Image superresolution



Saharia, C., Ho, J., Chan, W., Salimans, T., Fleet, D. J., & Norouzi, M. (2022). Image super-resolution via iterative refinement. IEEE PAMI

Applications

► Various image restoration problems



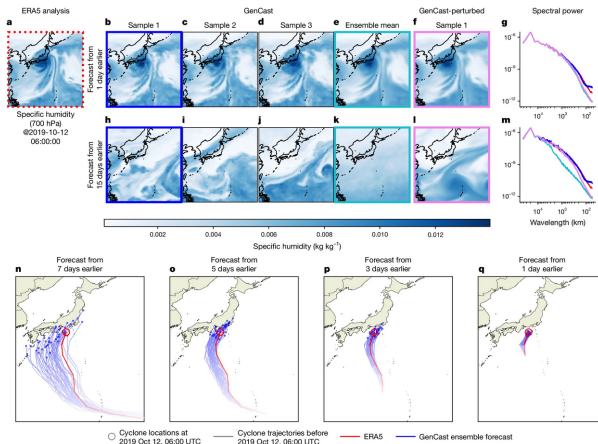
Pourya, M., Rawas, B. E., & Unser, M. (2025). FLOWER: A Flow-Matching Solver for Inverse Problems. arXiv preprint arXiv:2509.26287.

Applications

► Probabilistic forecasting given historical data

Fig. 2: Visualization of forecasts and tropical cyclone tracks.

From: [Probabilistic weather forecasting with machine learning](#)



a. The ERA5 analysis state²² for specific humidity at 700 hPa, at validity time 06 UTC, 12 October 2019, shows Typhoon Hagibis near the centre of the frame, hours before making landfall in Japan. b–d, Sample 1 (b), sample 2 (c) and sample 3 (d) GenCast forecast states, initialized one day earlier, show how the

Price, I., Sanchez-Gonzalez, A., Alet, F., Andersson, T. R., El-Kadi, A., Masters, D., ... & Willson, M. (2025). Probabilistic weather forecasting with machine learning. *Nature*, 637(8044), 84-90.

Related Tasks

In generative models the target distribution is known through samples only.

When instead the distribution is known only via its probability density, sampling or even calculating its expectation (or other integrals) is not straightforward.

Example:

$$\mathbb{E}[X] = \int_{\mathbb{R}^d} x p_X(x) dx \approx \frac{1}{M} \sum_{i=1}^M x_i$$

for M large enough and x_i are iid samples following the target distribution p_X

- ▶ Sampling algorithms (e.g. Markov Chain Monte Carlo) obtain i.i.d samples from the target. Example: the Unadjusted Langevin Algorithm:

$$x_{k+1} = x_k - \beta \nabla \log(p_X(x_k)) + \sqrt{2\beta} \xi_t$$

- ▶ Variational inference: approximate a distribution that is hard to manipulate by "the closest" distribution to the target within a parameterized class. This class is supposed to be "easy" to sample from:

$$\arg \min_{\theta} d(p_{\theta}, p_X)$$

for some family of tractable distributions $\{p_{\theta}\}$ and a similarity metric between distributions d .

Table of Contents

Generative models

Problem statement

Static models

Flow based models

The change of variables formula

Normalizing flows

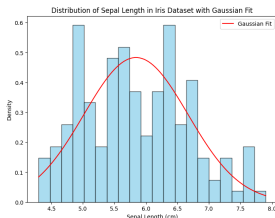
Flow matching

The continuity equation

Flow matching

Basic models

One example from the ML course: Gaussian generative model: each class follows a Gaussian distribution: $p(x|\mathcal{C}_k) = \mathcal{N}(\mu_k, \Sigma_k)$.



Gaussian approximation of a feature from the Iris dataset

1. Assume that the data are drawn from some parametric distribution for which sampling is easy
2. Estimate the parameters of this distribution from the data (e.g via Maximum Likelihood)
3. To generate new samples, just sample from the resulting distribution.

Exercise

Assume the data follow a GMM with K classes: $p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$. Once the model is trained, how do you sample a new data point? (This is called ancestral sampling)

Generative modelling as a neural mapping from a latent space to the target space

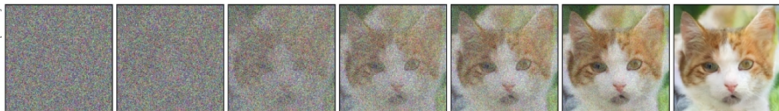
Pushforward models

Modern generative models are based on transforming samples from a known simple density (in most cases z is a Gaussian) via a learned function to match the target samples.

$$z \sim p_Z(z) \rightarrow x = f_\theta(z) \sim p_X(x)$$

- ▶ $z \in \mathbb{R}^{d'}$ is called a latent variable
- ▶ f is a function parameterized by θ that is going to be learned (using Neural Nets)

Heitz et al. [2023]



Questions:

1. For fixed θ , how do we sample from p_X ? (easy)
2. How do we make sure these samples x follow the same distribution as the data?
3. What kind of f_θ are allowed?
4. What is the pdf p_X of X ?

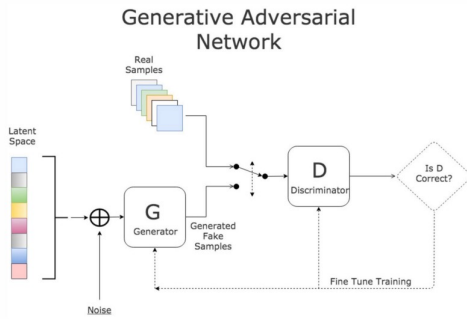
Generative modelling / what for ?

Generative modelling zoos

Generative Adversarial Networks

One of the first NN based answers to Q2 (samples follow the data distribution).
Two "adversarial" networks with conflicting objectives:

- ▶ One network D_{θ_d} trained to **discriminate** between training data (1) and fake data (0) (supervised classification)
- ▶ The other G_{θ_g} trained to **generate** data from a latent $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ as close as possible to real training data so as to fool D_{θ_d}



$$L(\theta) = \min_{\theta_g} \max_{\theta_d} [\mathbb{E}_{\mathbf{x} \sim p_{data}} \log(D_{\theta_d}(\mathbf{x})) + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \log(1 - D_{\theta_d}(G_{\theta_g}(\mathbf{z})))]$$

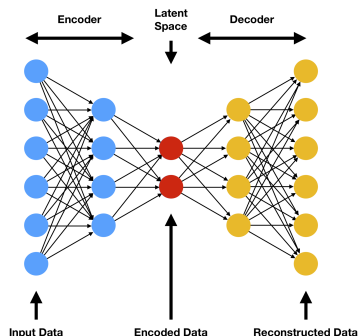
When the loss is stabilized: equilibrium, the generator samples from p_{data} .
Discriminator is no better than chance.

Variational Auto Encoders (VAE)

A Bayesian alternative to model $p(x|z)$ explicitly.

Reminder: Autoencoders (unsupervised learning)

- ▶ Objective: represent the data in a compressed latent space $z \in \mathbb{R}^{d'}$ (lower dimensional bottleneck) using an encoder $z = p_{\phi}(x)$
- ▶ Model is trained to maximize reconstruction from a decoder $\hat{x} = q_{\theta}(z)$:
$$L(\theta, \phi) = \mathbb{E}_{x \sim p_{data}} [||\hat{x} - x||^2]$$



Q: How to turn this into a generative model?

- ▶ The latent space z should be modeled probabilistically: enforce $z \sim \mathcal{N}(0, I)$.
- ▶ The decoder models $p_\theta(x|z)$ as a Normal distribution for each sample $p_\theta(z|x) \sim \mathcal{N}(\mu_\theta(x), \sigma_\theta^2(x))$.

This defines a Bayesian model for $p_\theta(x|z)$.

The goal is to maximize the likelihood of the data (assumed iid) under the model: $L(\theta, \phi) = \sum_{i=1}^N p(x = x_i)$. Problem:

$$p_\theta(x = x_i) = \int_{\mathbb{R}^{d'}} p_\theta(x = x_i|z)p(z)dz$$

is intractable because $p_\theta(x = x_i, z) = p_\theta(z|x = x_i)p_\theta(x = x_i)$.

Solution: find a lower bound of the likelihood involving a new distribution $q(z|x)$ (encoder). For any q :

$$\log p(x) \geq \mathbb{E}_{z \sim q} [\log p_{\theta}(x|z)] + KL(q||p)$$

where $KL(q||p) = -\mathbb{E}_{z \sim q} \left[\log \left(\frac{p(z)}{q(z|x)} \right) \right] \geq 0$ is the Kullback-Leibler divergence

This bound is called the Evidence Lower Bound (ELBO). Maximizing it over the parameters of q is a proxy to maximizing the likelihood.

Exercises:

1. Prove the previous inequality using Jensen's inequality applied to the log function.
2. Prove that the ELBO rewrites

$$\mathbb{E}_{z \sim q} [\log p_{\theta}(x|z)] + KL(q||p) = \log p_{\theta}(x) + KL(p_{\theta}(z|x)||q(z|x))$$

This shows that the ELBO is maximized when q is the posterior distribution. As it is intractable, we model $q_\phi(z|x) = \mathcal{N}(\mu_\phi(z), \sigma_\phi^2(z))$ and maximize the ELBO over $\{\theta, \phi\}$.

→ Variational inference using the ELBO:

$$\arg \max_{\theta, \phi} \sum_{i=1}^N \mathbb{E}_{z_i \sim q_{\phi_i}} [\log p_\theta(x_i | z_i)] + KL(q_{\phi_i} || p)$$

The first term is an expectation under the variational approximation of the decoder, and the second term has a closed form for Gaussian distributions. The loss becomes

$$\arg \max_{\theta, \phi} \sum_{i=1}^N \left(\mathbb{E}_{z_i \sim q_{\theta_i}} \left[-\frac{1}{2\sigma_\theta(z_i)} \|x_i - \mu_\theta(z_i)\|^2 \right] + \right. \\ \left. \frac{1}{2} \|\mu_\phi(x_i)\|^2 - d' \log(\sigma_\phi(x_i)) - d' \sigma_\theta(x_i)^2 \right)$$

Exercise

Derive the final form of the loss, from the first form.

Limitations of GANs and VAE

- ▶ Training GANs is notoriously hard (min/max problem)
- ▶ The Gaussian hypothesis + the compressed representation + the approximate loss limit VAE expressivity
- ▶ No explicit density of generated samples in both cases

Why would we want models with explicit densities?

1. Direct training via Maximum Likelihood

$$\arg \max_{\theta} \sum_{i=1}^N \log p_x(x_i)$$

2. Assess whether a new sample comes from the same distribution as the original data (outlier/anomaly detection)
3. Integration of conditioning in Bayesian models $p(x|y) \propto p(y|x)p(x)$, where y are observations. Useful in inverse problems

What constraints should be put on f ?

Back to the basic form of pushforward models:

$$z \sim p_Z(z) \rightarrow x = f_\theta(z) \sim p_X(x)$$

$$f_\theta : \mathbb{R}^{d'} \rightarrow \mathbb{R}^d$$

$$z \mapsto f_\theta(z) = x$$

Desirable properties for f

- ▶ If the support of p_X is all of $\mathbb{R}^d$, f must be surjective
- ▶ Any two z_1 and $z_2 \in \mathbb{R}^{d'}$ must lead to different samples: f must also be injective
- ▶ f should have some regularity: at least we would like "close" z not to map to very different x : we require f to be a homeomorphism, and even a diffeomorphism (for automatic differentiation).

As there exists no homeomorphism between Euclidean spaces of different dimensions, this also forces $d' = d$.

Table of Contents

Generative models

- Problem statement

- Static models

Flow based models

- The change of variables formula

- Normalizing flows

Flow matching

- The continuity equation

- Flow matching

The change of variable formula

Let $f : \mathbb{R}^{d'} \rightarrow \mathbb{R}^d$, $z \mapsto f(z) = x$ be a diffeomorphism, and Z a random vector with density p_z . Then, denoting by $J_{f^{-1}}(x)$ the Jacobian of f^{-1} at x , $X = f(Z)$ has a density p_x given by:

$$\forall x \in \mathbb{R}^d, p_x(x) = p_z(f^{-1}(x)) |\det(J_{f^{-1}}(x))|$$

Exercises:

1. Let $Z \sim \mathcal{N}(0, I_2)$. Compute and represent the density of $X = f(Z)$ where $f(z_1, z_2) = (x_1, x_2) = (z_1, z_2 + az_1^2)$ for $a \in \mathbb{R}$.
2. Show that for an analytically known density p^* , one can use the same principle for variational inference by minimizing $KL(p_x || p^*)$.

Normalizing Flows

This explicit likelihood suggests a new training strategy:

- ▶ Parameterize f by a NN f_θ
- ▶ Maximize the likelihood of the dataset under the model:

$$\arg \max_{\theta} \sum_{i=1}^N \log p_x(x_i) = \log(p_z(f_\theta^{-1}(x_i))) + \log \det(J_{f_\theta^{-1}}(x_i))$$

Exercise

Show that maximizing the likelihood is equivalent to minimizing $KL(p_{data} || p_x)$.

Computational cost

- ▶ $\frac{\partial f}{\partial \mathbf{z}}$ (and the gradient of the log likelihood wrt θ) is computed by automatic differentiation.
- ▶ We must guarantee that f_θ is invertible with tractable inverse (not the case for regular NN)
- ▶ Algorithmically, the computation of the determinant scales in $\mathcal{O}(d^3)$ in general.

This becomes costly in high dimensions, even prohibitive for e.g. large images ($d \sim 10^6 - 10^9$).

→ So this approach is impractical if implemented naively.

Table of Contents

Generative models

Problem statement

Static models

Flow based models

The change of variables formula

Normalizing flows

Flow matching

The continuity equation

Flow matching



These issues must be resolved by architectural constraints.

- ▶ As in regular NN, we stack layers of elementary invertible transformations

$$x = f(z) = f_{[1]} \circ f_{[2]} \circ \dots \circ f_{[L]}(z)$$

$$\det J_{f^{-1}} = \prod_{i=1}^n \det J_{f_{[i]}^{-1}},$$

"Elementary" means "Jacobian is easy and fast to compute".

Example of NF layers

- ▶ Autoregressive layers: for all i , $z_{[k+1]}^i$ only depends on $z_{[k]}^0, \dots, z_{[k]}^i$
- ▶ Coupling Layers: $z_{[k]}$ is partitioned into $[z_{[k]}^a, z_{[k]}^b]$ and $z_{[k+1]}^a$ depends on both $[z_{[k]}^a, z_{[k]}^b]$, while $z_{[k+1]}^b = z_{[k]}^b$

Exercise:

For autoregressive and coupling layers, show how computing the Jacobian scales better than $\mathcal{O}(d^3)$.



Residual Layers: $z_{[k+1]} = f_{\theta}(z_{[k]}) \triangleq z_{[k]} + g_{\theta}(z_{[k]})$. Invertible if g_{θ} is Lipschitz continuous with constant $L < 1$.

We have

$$\log \det J_{f_{\theta}} = \text{tr} \log(J_{f_{\theta}}) = \text{tr} \log(I + J_{g_{\theta}})$$

This can be expressed as a power series involving traces of powers of $J_{g_{\theta}}$.

Removes all the architectural constraints but the Lipschitz constraint.

Exercises

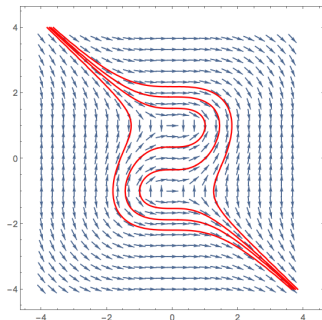
1. Prove that in this setting $y = x + g_{\theta}(x)$ is invertible and give an algorithm to compute the inverse.
Hint: write $x = T_y(x) \triangleq y - g_{\theta}(x)$ and use the Banach fixed point theorem.
2. Prove that Lipschitz constants behave multiplicatively for composition. Most usual activation functions are 1-Lipschitz. What constraint should every Linear layer satisfy then for a residual layer to be invertible?

Towards flow based models: Continuous Time Limit



The previous construction is reminiscent of the Euler discretization of the integration of the ODE

$$\frac{dz}{dt} = g_{\theta}(z), \quad z(0) = z_0$$



The flow $\Phi^t(z)$ is defined as the solution of this ODE at time t for $z(0) = z$. If the solution is uniquely defined for all t and a given $z(0)$, then the flow defines a bijection $\Phi^t : \mathbb{R}^d \rightarrow \mathbb{R}^d$ for all t .

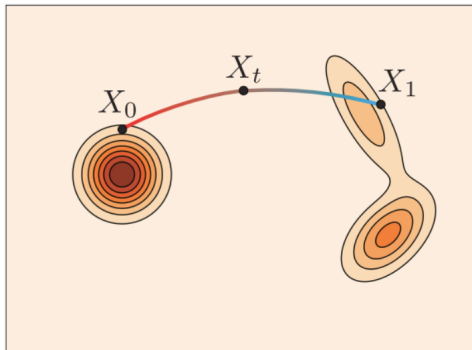
Equivalent to satisfying the Picard-Lindelöf (Cauchy-Lipschitz) theorem on the existence and uniqueness of ODE solutions: g_{θ} must be Lipschitz (any constant works!).

Eulerian vs Lagrangian views



If $z(0) \sim p_{z_0}$ then the dynamics *transport* the density into a new one p_{z_t}

- ▶ Eulerian view: we ask what the density at a fixed point in time and space is: $p_t(z) = p(z, t) = \Phi_{\#}^t p_0(z)$ where $\cdot_{\#}$ is the "pushforward" operator.
- ▶ Lagrangian view: we follow a particle's trajectory along the dynamics: sample z_0 , and $z(t)$ evolves following the ODE. If we are interested in the density, it is along a trajectory: $p_t(z(t)) = p(z(t), t)$.



Trajectories vs density path.

From <https://stevengong.co/notes/Flow-Matching>



Then, for $z_0 \sim p_{z_0}$, the dynamics defines a family of densities $p_{z_t} = p_t(\cdot)$ over $\mathbb{R}^d \times \mathbb{R}$ given by the following PDE (Liouville equation):

$$\frac{dp_t(z)}{dt} = -\text{div}(p_t(z)g_\theta(z))$$

→ This is the Eulerian view.

This translates into a Lagrangian view via the *instantaneous change of variables* equation on the log density following a trajectory $z(t)$:

$$\frac{D}{Dt} (\log(p_t(z(t)))) = -\text{tr}(J_{g_\theta}(z(t)))$$

$\frac{D}{Dt} f(z(t), t) = \nabla f(z(t), t) \cdot \dot{z}(t) + \partial_t f(z(t), t)$ is the *material derivative*

→ This provides the evolution of the log likelihood along a trajectory $z(t)$, and now the likelihood computation scales only in $\mathcal{O}(d^2)$.

Exercises

1. Derive the Liouville equation
2. Derive the "instantaneous change of variable" equation



Goal: for a data point $x = z(T)$ (in general $T = 1$), compute the $z(0)$ that generated it and the associated log likelihood $\log p(x) = \log p_T(z(T))$

Consider the following system of ODEs:

$$\frac{d}{dt} \begin{pmatrix} z(t) \\ s(t) \end{pmatrix} = \begin{pmatrix} g_\theta(z) \\ -\text{tr}(J_{g_\theta}(z(t))) \end{pmatrix}$$

with final condition $z(T) = x$ and $s(T) = 0$. Then for a given point x

1. Discretize the ODE with e.g. Euler's method (backwards in time to $T = 0$)
2. Compute $\log p(x) = \log p_0(z(0)) - s(0)$ ($s(0) \approx \int_T^0 -\text{tr}(J_{g_\theta}(z(t)))$)
3. Solve $\arg \min_{\theta} - \sum_{i=1}^N \log p(x_i)$ via gradient descent and autodiff.

Each likelihood computation requires one ODE integration, and multiple computations of the trace of $J_{g_\theta}(z(t))$. This can be costly in high dimensions.

Exercise

Consider $u \sim \mathcal{N}(0, I)$. Show that $\mathbb{E}[u^T J u] = \text{tr}(J)$, and that the sample estimator from m probes has variance $\mathcal{O}(1/m)$. Explain how this can help to further reduce the cost of computing the trace of the Jacobian.

Table of Contents

Generative models

- Problem statement

- Static models

Flow based models

- The change of variables formula

- Normalizing flows

Flow matching

- The continuity equation

- Flow matching

Flow-based generative modelling

Flow based models

Continuous NF are powerful (the NN to train is not very constrained), but the problem remains hard: the only supervision we have is given by the samples.

Flow matching methods seek to supervise the problem at all times, not just at final time $T = 1$.

Any ODE with a regular enough vector field:

$$\frac{dz}{dt} = v(z(t))$$

Translates into a Eulerian view into the Liouville (continuity) equation:

$$\frac{dp_t(z)}{dt} + \operatorname{div}(p_t(z)v(z)) = 0$$

The distribution is transported by the vector field $v(z)$, without mass being created. Notation: p_t is the pushforward of p_0 by the flow of the ODE:

$$p_t = \Phi_{\#}^t p_0$$

Flow matching

Define a path p_t transporting the base distribution (p_0) to the data distribution ($p_1 = p_{data}$), and find a *matching* vector field to come back at the level of trajectories: the resulting ODE can be used to sample.

Table of Contents

Generative models

- Problem statement

- Static models

Flow based models

- The change of variables formula

- Normalizing flows

Flow matching

- The continuity equation

- Flow matching**

Inverting the continuity equation

But how can we derive a vector field from a given continuity equation?

Consider the path $p_t = \mathcal{N}(\mu_t, I)$ where $\mu_t = (1 - t)\mu_0 + t\mu_1$.

A simple (constant) vector field corresponding to this path is given by

$$v_t(z) \equiv v = \mu_1 - \mu_0.$$

All points are simply translated by v : $z(t) = z_0 + tv$. But now define

$$v_t^\omega(z) = \omega J(z + (1 - t)\mu_0 + t\mu_1), \quad \text{where} \quad J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

$$z^\omega(t) = \mu_0 + tv + R(\omega t)(z_0 - \mu_0), \quad \text{with} \quad R(\omega t) = \begin{pmatrix} \cos(\omega t) & -\sin(\omega t) \\ \sin(\omega t) & \cos(\omega t) \end{pmatrix}$$

Both sets of trajectories correspond to the same path p_t !

Even in this simple case, there is an infinity of solutions! So the problem of finding a suitable vector field is an *ill-posed inverse problem* in general.

Exercise

Prove that the distribution of $z^\omega(t)$ is given by p_t when $z_0 \sim p_0$.

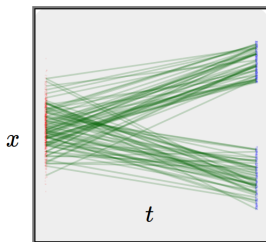
Flow matching

For a given path p_t , actually any v_t suffices as long as it is tractable (can be obtained numerically). We define a path the following way:

1. Sample independently a point from the base distribution z_0 and a data point z_1 . Define

$$z_t = (1 - t)z_0 + tz_1 = T_t(z_0, z_1)$$

This is an interpolation at the level of trajectories, but it cannot be used to sample new points.



From: <https://dl.heeere.com/conditional-flow-matching/blog/conditional-flow-matching/>

2. Define p_t as the law of z_t when $[z_0, z_1]$ follow the *independent coupling* $p_0 \otimes p_{data}$. $p_t =$, i.e. $p_t = T_{t\#}(p_0 \otimes p_1)$ We check that this interpolates between the base and data distributions.

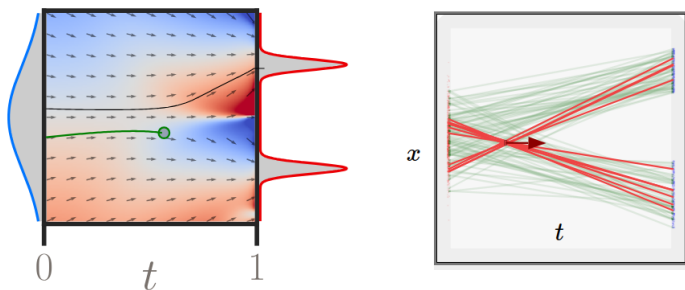
Conditional expectation

For this construction, a suitable vector field is given by

$$v_t(z) = \mathbb{E}[z_1 - z_0 | (1 - t)z_1 + tz_0 = z]$$

We can show that this v_t satisfies the continuity equation for the family p_t .

This conditional expectation says that the vector field is given by the average of the directions of all (straight) trajectories linking a base point and a data point that pass through z at time t .



Vector field used to sample (left), Realization as a conditional expectation (right)

Regression Loss

However this formula does not tell us how to compute the vector field v_t in practice. But remember that conditional expectation is linked to L_2 norm regression: for random variables X and Y with finite second order moments, we have

$$\mathbb{E}[Y|X] = \min_g \|Y - g(X)\|_{L_2}^2 = \min_g \mathbb{E}_{(X,Y)} \|Y - g(X)\|^2$$

In our context, $v_t(z) = \mathbb{E}[z_1 - z_0 | (1-t)z_1 + tz_0 = z]$ translates into

$$\arg \min_{(v_t)_t} \int_0^1 \int_{\mathbb{R}^d \times \mathbb{R}^d} \|v_t((1-t)z_1 + z_0) - (z_1 - z_0)\|^2 p_0(z_0) p_1(z_1) dz_0 dz_1 dt$$

This is also saying that this v_t is the best predictor (in the L_2 sense) of the velocity of the probability path given the position.

To turn this fact into a loss for a generative model, parameterize $v_\theta(z, t) = v_t^\theta(z)$ by a neural network and compute the integrals via Monte Carlo sampling

$$\arg \min_{\theta} \mathbb{E}_{t \sim \mathcal{U}[0,1], (z_0, z_1) \sim p_0 \otimes p_{data}} \|v_t^\theta((1-t)z_1 + z_0) - (z_1 - z_0)\|^2$$

Sampling and likelihood computations

The vector field is parameterized by a *time-dependent* neural net $v_\theta(z, t)$ (see lab)

→ This must be considered in the architecture with embedding strategies, depending on the complexity of the data and target:

- ▶ Concatenating z and t in the input
- ▶ Layer wise time injection
- ▶ Positional encoding and Fourier Features
- ▶ Attention Layers

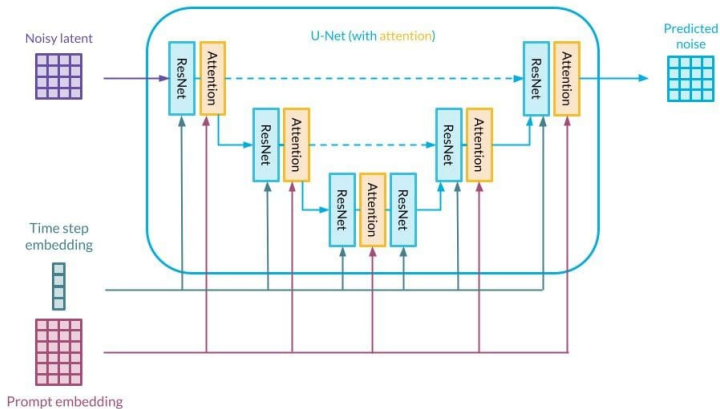
Once trained, to sample a new point, sample from z_0 and solve the ODE on $[0, 1]$ with this initial condition:

$$\frac{dz}{dt} = v_t(z(t))$$

Likelihoods can be obtained exactly as in Continuous Normalizing Flows using the instantaneous change of variable formula.

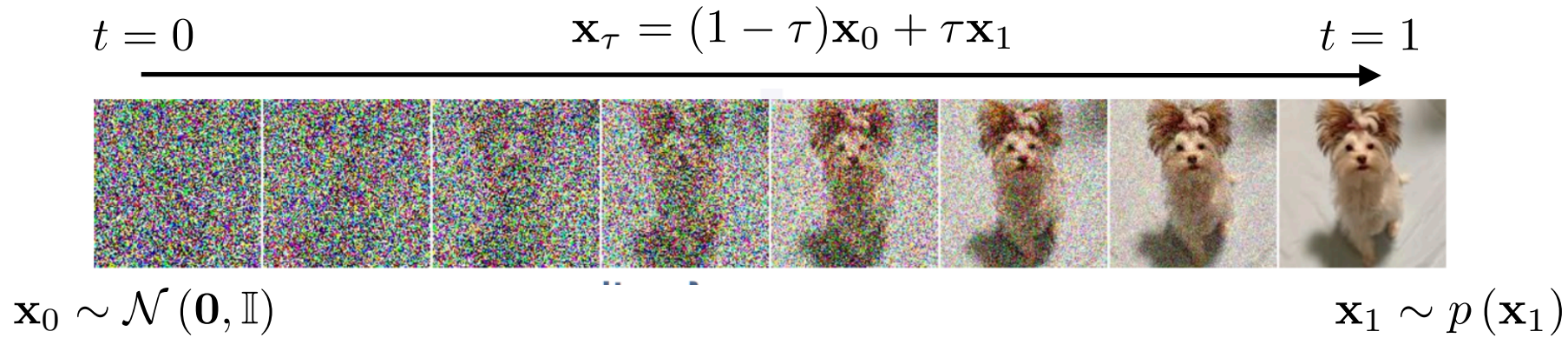
Time dependent vector fields for image data

For image data, the vector field is typically modeled by a *time-dependent* U-Net



Flow matching summary

Flow matching path from a Gaussian space to the targeted physical space



Associated flow-based sampling ODE: $\frac{d\mathbf{x}_\tau}{d\tau} = \mathbb{E}[\mathbf{x}_1 - \mathbf{x}_0 | \mathbf{x}_\tau]$

Unets as state-of-the-art approximators: $\mathcal{U}_{net}(\mathbf{x}_\tau, \tau) = \mathbb{E}[\mathbf{x}_1 - \mathbf{x}_0 | \mathbf{x}_\tau]$

Different flow matching parameterizations

Diffusion models are another popular classe of generative models. They can be related to flow matching:

- Instead of specifying p_t directly, we define a Stochastic Differential Equation (SDE) (an ODE with "noise")

$$dz = f(z, t)dt + g(t)dW_t$$

whose marginals are p_t . For instance: $dz = \exp(t)dW_t$ pushes the data distribution towards a Gaussian (progressively adds noise to interpolate between p_{data} and a Gaussian for large enough final times.)

- We can show that the ODE:

$$\frac{dz}{dt} = v_t(z(t)) = f(z, t) - \frac{1}{2}g(t)\nabla_z \log(p_t(z))$$

satisfies the continuity equation for the path p_t .

- Then, instead of learning directly v_t , we use this specific form and learn $s_\theta(z, t) = \nabla_z \log(p_t(z))$ via score-matching techniques

Summary

Generative models range from simple parametric models (Gaussian models) to modern flow-based approaches.

- ▶ Use the expressivity of neural networks
- ▶ Obtain likelihoods without constraining the architecture too much
- ▶ Discrete NF have a lot of constraints in their architectures, limiting expressivity
- ▶ CNF have more tractable likelihoods and less constrained vector fields
- ▶ Flow matching supervizes training at all time steps, making the problem easier
- ▶ In Flow matching, an unconstrained (but time dependent) neural net has to be parameterized
- ▶ Diffusion models can be seen as a special case of Flow matching

References

- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. *NeurIPS 2014*, 27.
- Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. *ICLR 2014*
- Papamakarios, G., Nalisnick, E., Rezende, D. J., Mohamed, S., & Lakshminarayanan, B. (2021). Normalizing flows for probabilistic modeling and inference. *JMLR*, 22(57), 1-64.
- Behrmann, J., Grathwohl, W., Chen, R. T., Duvenaud, D., & Jacobsen, J. H. (2019, May). Invertible residual networks. In *ICML (pp. 573-582)*. PMLR.
- Grathwohl, W., Chen, R. T., Bettencourt, J., Sutskever, I., & Duvenaud, D. (2019). Ffjord: Free-form continuous dynamics for scalable reversible generative models. *ICLR 2019*.
- Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M., & Le, M. Flow Matching for Generative Modeling. In *The Eleventh International Conference on Learning Representations*.
- Peyré, G. (2025). Optimal and Diffusion Transports in Machine Learning. *arXiv preprint arXiv:2512.06797*.
- Gagneux, A., Martin, S., Emonet, R., Bertrand, Q., Massias, M., ICLR Blogpost 2025 on Flow Matching: <https://dl.heeere.com/conditional-flow-matching/blog/conditional-flow-matching/>
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR 2020*.